A

Major Project

On

ENSEMBLE FEATURE LEARNING FOR ACCURATE RECOGNITION OF INDUSTRIAL TOOLS IN WORKSHOP ENVIRONMENTS

(Submitted in partial fulfilment of the requirements for the award of Degree)

BACHELOR OF TECHNOLOGY

In

INFORMATION TECHNOLOGY

By

K Sai Samhitha (227R1A1296)

A Nikhil Goud (227R1A1266)

K Lokesh (227R1A12A2)

Under the Guidance of

Mrs. M Sivajyothi

Assistant Professor

DEPARTMENT OF INFORMATION TECHNOLOGY

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NAAC, NBA, Permanently Affiliated to JNTUH, Approved by AICTE, New Delhi)

Recognized Under Section 2(f) & 12(B) of the UGCAct.1956,

Kandlakoya (V), Medchal Road, Hyderabad-501401.

May, 2026.

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that the project entitled **"ENSEMBLE FEATURE LEARNING FOR ACCURATE RECOGNITION OF INDUSTRIAL TOOLS IN WORKSHOP ENVIRONMENTS"** being submitted by **K. Sai Samhitha (227R1A1296), A. Nikhil Goud (227R1A1266) & K. Lokesh (227R1A12A2)** in partial fulfilment of the requirements for the award of the degree of B.Tech in Information Technology to the Jawaharlal Nehru Technological University Hyderabad, during the year 2025-26.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Mrs. M . Sivajyothi
Assistant Professor
INTERNAL GUIDE

Head of the Department

Dr. A. Raji Reddy
DIRECTOR

Signature of External Examiner

Submitted for viva voice Examination held on _____

ACKNOWLEDGEMENT

We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project, we take this opportunity to express our profound gratitude and deep regard to our guide **Mrs. M. Sivajyothi,** Assistant Professor for her exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by him shall carry us a long way in the journey of life on which we are about to embark.

We take this opportunity to extend our heartfelt appreciation to the Project Review Committee (PRC) Coordinators— **Dr. J. Narasimharao, Mrs. K. Supriya Suhasini, Mr. D. Krishna Kumar, Mr. G. Shankar, Mrs. M. SivaJyothi and Ms. G. Menaka** —for their unwavering support, insightful guidance, and valuable inputs, which played a crucial role in steering this project through its various stages.

Our sincere appreciation also goes to our **Head of the Department**, for encouragement and continuous support in ensuring the successful completion of our project.

We are deeply grateful to **Dr. A. Raji Reddy**, Director, for his cooperation throughout the course of this project. Additionally, we extend our profound gratitude to Sri. **C. Gopal Reddy**, Chairman, Smt. **C. Vasantha Latha**, Secretary and Sri. **C. Abhinav Reddy**, Vice-Chairman, for fostering an excellent infrastructure and a conducive learning environment that greatly contributed to our progress.

We also acknowledge and appreciate the guidance and assistance provided by the faculty and staff of **CMR Technical Campus**, whose contributions have been invaluable in bringing this project to fruition.

Lastly, we sincerely thank our families for their unwavering support and encouragement. We also extend our gratitude to the teaching and non-teaching staff of CMR Technical Campus for their guidance and assistance. Their contributions, along with the support of everyone who helped directly or indirectly, have been invaluable in the successful completion of this project.

K. Sai Samhitha (227R1A1296)
A Nikhil Goud (227R1A1266)
K Lokesh (227R1A12A2)

VISION AND MISSION

INSTITUTE VISION:

To Impart quality education in serene atmosphere thus strive for excellence in Technology and Research.

INSTITUTE MISSION:

1. To create state of art facilities for effective Teaching- Learning Process.

2. Pursue and Disseminate Knowledge based research to meet the needs of Industry & Society.

3. Infuse Professional, Ethical and Societal values among Learning Community.

DEPARTMENT VISION:

To produce technically proficient and skilled graduates in Information Technology and to prepare them to meet the needs of industry and society in a holistic learning environment.

DEPARTMENT MISSION:

1. To prepare students with the essential knowledge and contemporary abilities for solving emerging technological problems.

2. To train the students on cutting – edge computing facilities with industry interaction to enhance in practical exposure to real world situations.

3. To enhance the leadership qualities and team spirit with social values to serve the society.

ABSTRACT

In recent years, industrial automation and smart manufacturing have witnessed rapid growth, with global reports indicating that over 65% of workshops now rely on visual inspection systems to improve productivity and reduce operational errors, yet manual tool identification still contributes to nearly 30% of workflow inefficiencies and misclassification-related downtime. In this context, this work focuses on the automated recognition of industrial tools in workshop environments using intelligent vision-based techniques. The problem addressed in this study is the inaccuracy, time consumption, and subjectivity associated with traditional manual tool identification systems, where human operators rely on visual inspection and experience to classify tools. Such manual systems are prone to fatigue, inconsistency, limited scalability, and high error rates under varying lighting, background clutter, and tool similarity conditions, making them unsuitable for real-time industrial applications. The limitations of the manual approach motivate the need for an automated, robust, and scalable solution capable of delivering high accuracy with minimal human intervention. To address these challenges, a proposed system is developed using deep feature learning and ensemble classification techniques. The system employs InceptionResNetV2 as a deep convolutional neural network for powerful feature extraction from tool images, followed by multiple machine learning classifiers for comparison, including Decision Tree Classifier and Perceptron Classifier as existing baseline models. Furthermore, a hybrid ensemble model is introduced, integrating a Deep Neural Network (DNN) for probability-based feature transformation and a Random Forest Classifier for final decision-making, thereby enhancing classification robustness and generalization. Performance evaluation is carried out using accuracy, precision, recall, F1-score, ROC curves, AUC scores, and confusion matrices. The proposed hybrid DNN–Random Forest approach demonstrates superior performance over existing methods, effectively solving the identified problem by providing an accurate, automated, and real-time industrial tool recognition system suitable for smart workshop environments.

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
Figure 5.1	Uploading dataset interface after admin login/signup	36
Figure 5.2	InceptionResNetV2 feature extraction completed	36
Figure 5.3	Confusion matrix and ROC Curve obtained using DTC	37
Figure 5.4	Confusion matrix and ROC Curve obtained using KNN	37
Figure 5.5	Confusion matrix and ROC Curve obtained using perceptron classifier	38
Figure 5.6	Confusion matrix and ROC Curve obtained using Hybrid DNN with RF classifier +	38
Figure 5.7	Prediction on test data using hybrid DNN with RF model	39

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
Table 6.2.1	Uploading Data Set	41
Table 6.2.2	Classification	42
Table 6.2.3	Prediction of Industrial tools	43

TABLE OF CONTENTS

ABSTRACT **i**
LIST OF FIGURES **ii**
LIST OF TABLES **iii**
1. INTRODUCTION **1**
1.1 PROJECT PURPOSE 2
1.2 PROJECT FEATURES 2
2. LITERATURE SURVEY **3**
2.1 REVIEW OF RELATED WORK 4
2.2 DEFINITION OF PROBLEM STATEMENT 5
2.3 EXISTING SYSTEM 5
2.4 PROPOSED SYSTEM 7
2.5 OBJECTIVES 8
2.6 HARDWARE & SOFTWARE REQUIREMENTS 8
2.6.1 HARDWARE REQUIREMENTS 8
2.6.2 SOFTWARE REQUIREMENTS 8
3. SYSTEM ARCHITECTURE & DESIGN **9**
3.1 PROJECT ARCHITECTURE 9
3.2 DESCRIPTION 10
3.3 USE CASE DIAGRAM 11
3.4 CLASS DIAGRAM 12
3.5 DATA DLOW DIAGRAM 13
3.6 ACTIVITY DIAGRAM 14
3.7 SEQUENCE DUAGRAM 15
4. IMPLEMENTATION **16**
4.1 ALGORITHMS USED 17
4.2 SAMPLE CODE 19
5. RESULTS & DISCUSSION **36**
6. VALIDATION **40**
6.1 INTRODUCTION 40
6.2 TEST CASES 41
6.2.1 UPLOADING DATASET 41
6.2.2 CLASSIFICATION 42
6.2.3 PREDICTION OF INDUSTRIAL TOOL 43
7. CONCLUSION & FUTURE ASPECTS **44**
7.1 PROJECT CONCLUSION 44
7.2 FUTURE ASPECTS 45
8. BIBLIOGRAPHY **46**
8.1 REFERENCES 46
8.2 GITHUB LINK 47

1.INTRODUCTION

1.INTRODUCTION

Industrial workshops require efficient and reliable systems for identifying and managing tools to ensure safety, productivity, and operational efficiency. In modern manufacturing environments, manual identification of tools is time-consuming and prone to human errors, especially in complex workshop conditions where tools may appear similar or may be partially hidden. The proposed project, titled "Ensemble Feature Learning for Accurate Recognition of Industrial Tools in Workshop Environments," focuses on developing an intelligent automated system capable of recognizing industrial tools using advanced image-based machine learning techniques.

The system utilizes ensemble feature learning by combining Convolutional Neural Networks (CNN) with classical machine learning classifiers such as Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Random Forest to improve recognition accuracy even under challenging conditions like cluttered backgrounds, lighting variations, or partial occlusions. This technology has important applications in smart manufacturing environments where automated tool recognition can support inventory management, assist industrial robots in selecting the correct tools, and enhance training systems for new workers learning tool identification.

The rapid development of digital technologies, data acquisition systems, and communication networks over the past decade has also contributed to the growth of concepts such as the Internet of Things (IoT) and Industry 4.0, which aim to create interconnected and intelligent industrial systems. However, implementing IoT-based solutions in traditional workshops remains challenging due to limitations in existing machine tools, lack of integrated sensors, and restricted communication capabilities. Many machines rely on Computer Numerical Control (CNC) systems or Programmable Logic Controllers (PLC) to extract operational data, and therefore additional data acquisition platforms are often required to monitor industrial processes effectively.

Machine learning techniques provide powerful solutions for analyzing industrial data, particularly when dealing with incomplete, noisy, or unbalanced datasets commonly found in real workshop environments. These algorithms can extract useful patterns, reduce redundant features, and generalize models to new operating conditions without requiring extensive experimental trials. By integrating machine learning with industrial automation technologies, the proposed system aims to provide an efficient and intelligent solution for automated industrial tool recognition, contributing to the advancement of smart manufacturing and Industry 4.0 applications.

1.1 PROJECT PURPOSE

The main purpose of this project is to design and develop an automated **Industrial Tool Recognition System** using image processing and machine learning techniques. The system aims to accurately identify and classify different industrial tools from images. It reduces the need for manual identification of tools in workshop environments. The project uses deep learning models to extract important visual features from tool images. These features are then used by machine learning classifiers to recognize tool categories. The system helps minimize human errors during tool identification. It also improves efficiency and productivity in industrial workplaces. The project demonstrates how artificial intelligence can be applied to real-world manufacturing problems. In addition, it provides a scalable solution that can be extended to recognize more tools in the future. Overall, the purpose of this project is to support automation and smart manufacturing through intelligent tool recognition.

1.2 PROJECT FEATURES

The proposed **Industrial Tool Recognition System** includes several important features that help in accurately identifying and classifying industrial tools using advanced machine learning techniques. The system automatically recognizes tools from input images by applying image processing and deep learning methods. A pretrained deep learning model, **InceptionResNetV2**, is used for extracting high-level visual features such as edges, shapes, and textures from the tool images, which improves the quality of classification. The system supports multiple machine learning classifiers including Decision Tree, K-Nearest Neighbors (KNN), and Perceptron to analyze the extracted features and categorize the tools. In addition, a hybrid model combining Deep Neural Network (DNN) and Random Forest is implemented to enhance prediction accuracy and robustness. The system is capable of performing multi-class classification to identify different types of tools such as hammer, pliers, gasoline can, and pebble. To evaluate the effectiveness of the models, performance metrics such as accuracy, ROC curve, and confusion matrix are used. The system also supports prediction on unseen test images, demonstrating its ability to generalize to new data. Furthermore, the design of the system allows easy expansion by adding more tool categories in the future. Overall, the project provides a reliable and scalable framework for automated industrial tool recognition.

2.LITERATURE SURVEY

2. LITERATURE SURVEY

Recent advancements in machine learning and deep learning have significantly improved the development of intelligent systems for industrial automation, monitoring, and recognition tasks. Several researchers have explored ensemble learning techniques to improve prediction accuracy and robustness. Garrido-Labrador JL et al. [6] evaluated various algorithmic techniques for optimizing machining processes using real industrial datasets and found that ensemble models, particularly Rotation Forest ensembles, achieved superior performance in classifying machining center states and predicting motor thermal behavior. Odeh A et al. [7] proposed an ensemble deep learning framework combining CNN, LSTM, and GRU models with a voting mechanism, achieving extremely high classification accuracies of 99.7% and 99.6%. Goodarzi P et al. [8] introduced a multi-sensor ensemble framework integrated with AutoML to handle domain shifts and variability in sensor data while reducing training costs. Jurek A et al. [9] presented a cluster-based ensemble method for activity recognition in smart environments, demonstrating its effectiveness using both numeric and binary sensor data. Yucesoy E et al. [10] developed ensemble models using classifiers such as KNN, SVM, Logistic Regression, Random Forest, and Extra Trees, achieving an accuracy of 97.41% using stacking techniques. Lai W-I et al. [11] proposed ensemble recurrent neural network models to improve the accuracy of gas concentration detection using low-cost sensors, while Ma Y et al. [12] developed an American Sign Language recognition system using multiple CNN architectures combined with an accuracy-weighted voting strategy. Zhao H et al. [13] applied ensemble learning for cybersecurity anomaly detection by analyzing cyber-attack propagation patterns and dynamically updating training weights. Du K-L et al. [14] provided a comprehensive review of ensemble learning methods including bagging, boosting, and random forests, highlighting their ability to reduce prediction errors and improve model generalization. Similarly, Alshazly H et al. [15][16] implemented ensemble deep CNN models for ear recognition using VGG-based architectures and achieved improved identification performance. De Zarzà I et al. [17] investigated ensemble approaches such as bagging, stacking, and cascading neural networks for predictive modeling and demonstrated improved prediction accuracy in healthcare applications. Ku Abd. Rahim KN et al. [18] applied ensemble classifiers to human activity recognition using smartphone sensor data, while Alalwany E et al. [19] and Alkadi S et al. [20] used ensemble methods for intrusion detection systems in IoT networks to enhance detection accuracy and system robustness. Haider T et al. [21] and Saha J et al. [22] explored ensemble learning techniques for human activity recognition using wearable and smartphone sensors. Przybyś-Małaczek A et al. [23] compared deep learning and machine learning models for tool state

recognition in CNC milling processes, highlighting the importance of feature selection and preprocessing. Bhattacharya D et al. [24] proposed an ensemble deep learning model combining CNN and LSTM architectures for activity recognition, achieving high accuracy across multiple datasets. Lee S et al. [25] further introduced a multi-stage ensemble approach for machine fault diagnosis using convolutional networks and classifier ensembles to improve detection accuracy. Overall, these studies demonstrate that ensemble learning and deep learning techniques significantly enhance prediction accuracy, reliability, and performance in various domains such as industrial monitoring, activity recognition, cybersecurity, and sensor-based systems, motivating the development of intelligent automated tool recognition systems for industrial workshop environments.

2.1 REVIEW OF RELATED WORK

Many researchers have explored the application of machine learning, deep learning, and ensemble learning techniques for classification and recognition tasks across various domains such as industrial systems, activity recognition, cybersecurity, healthcare, and sensor-based monitoring. Previous studies have shown that traditional machine learning algorithms like Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Random Forest (RF), and Logistic Regression (LR) are widely used for classification problems due to their simplicity and effectiveness. However, their performance may decrease when dealing with complex datasets, high-dimensional features, or unbalanced data.

Recent advancements in deep learning have introduced powerful models such as Convolutional Neural Networks (CNN), Long Short-Term Memory (LSTM), and Gated Recurrent Units (GRU), which can automatically extract meaningful features from data. These models have demonstrated significant improvements in image recognition, sensor data analysis, and pattern classification tasks. In industrial environments, deep learning techniques have been used for tool condition monitoring, fault diagnosis, and activity recognition. Despite their high accuracy, single deep learning models may suffer from overfitting and computational complexity.

To overcome these limitations, several studies have focused on ensemble learning approaches that combine multiple machine learning or deep learning models to improve performance. Ensemble techniques such as bagging, boosting, stacking, and voting strategies have been shown to enhance classification accuracy, stability, and generalization capability. By integrating predictions from multiple models, ensemble systems can reduce prediction errors and provide more reliable results compared to individual classifiers.

In the context of industrial applications, previous research has demonstrated that ensemble models are effective in analyzing sensor data, predicting machine conditions, and improving process optimization. These systems have been successfully applied in manufacturing environments, smart sensors, and automated monitoring systems. However, many existing works focus primarily on sensor-based data analysis rather than image-based recognition of industrial tools.

Therefore, there is still a need for intelligent systems capable of accurately recognizing industrial tools from visual data in workshop environments. Developing a system that combines deep feature extraction with ensemble classification techniques can provide improved recognition accuracy and robustness under real-world conditions such as varying lighting, cluttered backgrounds, and tool similarities. The proposed project aims to address this gap by implementing an ensemble feature learning framework for accurate recognition of industrial tools using image-based machine learning methods.

2.2 DEFINATION OF PROBLEM STATEMENT

In industrial workshop environments, accurate identification of tools is essential for maintaining efficiency, safety, and productivity. However, manual identification of industrial tools can be time-consuming and prone to human errors, especially when tools appear visually similar or are placed in cluttered environments. Existing systems often rely on limited datasets or single machine learning models, which may not provide high accuracy under real-world conditions. Therefore, there is a need to develop an automated and intelligent tool recognition system that can accurately classify industrial tools using image-based analysis. The proposed system aims to address this problem by applying ensemble feature learning techniques to improve recognition accuracy and reliability.

2.3 EXISTING SYSTEM

In traditional workshop environments, tool recognition and management are primarily manual or semi-automated through barcode or RFID-based systems. Workers typically identify and retrieve tools by visually inspecting shelves or using handheld scanners to detect tagged tools. While these methods are simple to implement, they come with significant limitations. Human error is common during tool selection, and time is wasted on repetitive searches. Barcode or RFID systems, although somewhat efficient, rely heavily on the physical integrity of tags, which may wear out or get detached in industrial settings. Additionally, such systems lack the ability to visually interpret the tool's appearance, making them ineffective in recognizing untagged or

visually similar tools. Rule-based image processing methods also fail under variable lighting, angle, or background noise. Overall, traditional systems are rigid, error-prone, and inefficient for real-time, scalable, and intelligent tool recognition tasks in modern industrial workshops.

Traditional System Description

Traditional tool recognition in workshops largely depends on manual processes or primitive automation methods, including:

- **Human Supervision:** Workers visually identify and select tools from storage racks or boxes.
- **Barcode/RFID Systems:** Tools are tagged with barcodes or RFID chips, scanned during issuance and return.
- **Fixed Rule-Based Systems:** Predefined rule-based detection using basic image processing (edge detection, shape templates) for tool identification.

Limitations of Traditional Systems

- **Error-Prone:** Human oversight often results in incorrect tool selection or misplacement.
- **Time-Consuming:** Manual checking and tool retrieval slow down overall workflow.
- **No Visual Intelligence:** Barcode/RFID systems cannot "see" tools or understand their visual features.
- **Scalability Issues:** Increasing tool types or quantities burdens barcode management and human memory.
- **Dependency on Tags:** If a barcode or RFID tag is damaged or lost, the tool becomes untraceable.
- **Static Detection Methods:** Rule-based systems fail to adapt to tool variations, orientations, or lighting conditions.
- **No Real-Time Feedback:** These systems lack the capability to detect errors or provide immediate classification insights.

2.4 PROPOSED SYSTEM

The proposed system focuses on developing an intelligent industrial tool recognition system using ensemble feature learning techniques. In this system, images of different industrial tools are collected and preprocessed to improve their quality and remove noise. A Convolutional Neural Network (CNN) is used to automatically extract important visual features from the tool images. These extracted features are then provided to multiple machine learning classifiers such as Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Random Forest for classification. The predictions from these classifiers are combined using an ensemble learning approach to improve the overall recognition accuracy. The system is designed to work efficiently even in challenging workshop environments where tools may appear in different orientations, lighting conditions, or cluttered backgrounds.

Advantages of the Proposed System

- **Higher Accuracy –** The ensemble learning approach improves classification accuracy by combining the strengths of multiple classifiers.
- **Automatic Feature Extraction –** CNN automatically extracts meaningful visual features without manual feature engineering.
- **Reduced Human Error –** The automated system reduces dependency on manual tool identification.
- **Robust Performance –** The system can recognize tools even under complex workshop conditions such as different lighting and backgrounds.
- **Improved Productivity –** Faster and more reliable tool recognition helps improve efficiency in industrial operations.
- **Scalability –** The system can be extended to recognize more types of tools by adding new training data.

2.5 OBJECTIVES

- **Develop an Automated Tool Recognition System**
  To develop an automated system that can accurately recognize and classify industrial tools using image processing techniques.
- **Extract Important Visual Features Using CNN**
  To extract meaningful visual features from tool images using Convolutional Neural Networks.
- **Apply Ensemble Learning Methods**
  To improve classification accuracy by combining multiple machine learning classifiers.
- **Evaluate Model Performance**
  To analyze and compare the performance of different machine learning models.
- **Support Industrial Automation**
  To assist workshop environments by enabling efficient and reliable tool recognition.

2.6 HARDWARE & SOFTWARE REQUIREMENTS

2.6.1 HARDWARE REQUIREMENTS:

Hardware interfaces specifies the logical characteristics of each interface between the software product and the hardware components of the system. The following are some hardware requirements,

- Processor : Intel i5
- Hard Disk : 1TB
- RAM : 8GB

2.6.2 SOFTWARE REQUIREMENTS:

Software Requirements specifies the logical characteristics of each interface and software components of the system. The following are some software requirements,

- Front end Technologies : Tkinter
- Back end Technologies : Python 3.7.6
- Operating System : Windows 10
- Database : MySQL

3.SYSTEM ARCHITECTURE & DESIGN

3. SYSTEM ARCHITECTURE & DESIGN

Project architecture refers to the structural framework and design of a project, encompassing its components, interactions, and overall organization. It provides a clear blueprint for development, ensuring efficiency, scalability, and alignment with project goals. Effective architecture guides the project's lifecycle, from planning to execution, enhancing collaboration and reducing complexity.

3.1 PROJECT ARCHITECTURE

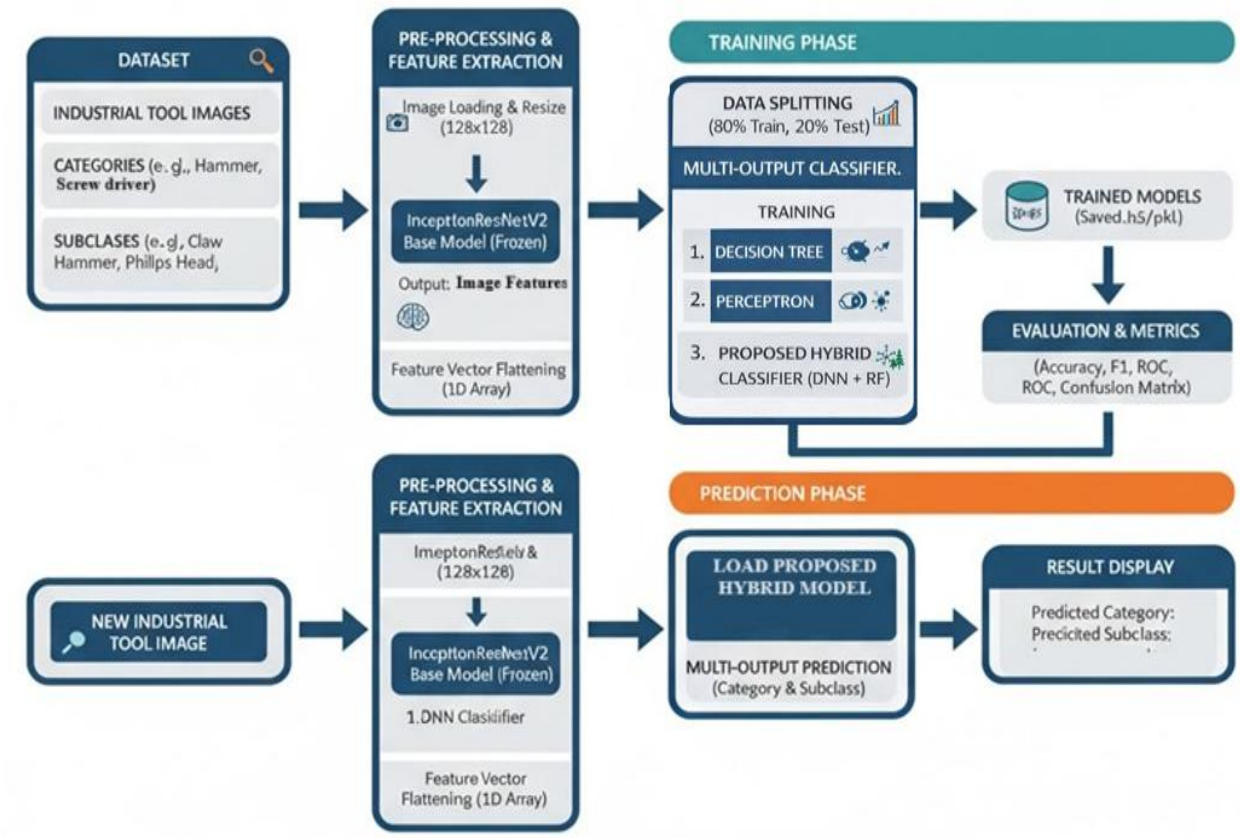


Figure 3.1: Project Architecture of Ensemble Feature Learning for Accurate Recognition of Industrial Tools in Workshop Environments

3.2 DESCRIPTION

Dataset: This block contains the collection of industrial tool images used for training and testing the model. The dataset includes different tool categories such as hammer and screwdriver along with their subclasses.

Pre-Processing & Feature Extraction (Training Phase): In this stage, the input images are loaded and resized to a fixed size of 128×128 pixels. The images are then passed through the InceptionResNetV2 base model to extract important visual features.

Data Splitting: The dataset is divided into two parts: training data and testing data. Typically, 80% of the data is used for training the model and 20% is used for evaluating its performance.

Multi-Output Classifier Training: Different classifiers such as Decision Tree, Perceptron, and the proposed hybrid model are trained using the extracted feature vectors. These classifiers learn patterns to recognize tool categories and subclasses.

Trained Models: After the training process, the trained models are saved for future predictions. These models contain the learned knowledge required for identifying industrial tools.

Evaluation & Metrics: The trained models are evaluated using performance metrics such as accuracy, F1-score, ROC curve, and confusion matrix. This helps measure the effectiveness and reliability of the classification models.

New Industrial Tool Image: This block represents the input image provided by the user during the prediction stage. The image is a new tool image that the system has to recognize.

Pre-Processing & Feature Extraction (Prediction Phase): The new input image undergoes the same preprocessing steps such as resizing and feature extraction. The InceptionResNetV2 model extracts the feature vector required for prediction.

Load Proposed Hybrid Model: The previously trained hybrid model is loaded into the system. This model is responsible for performing the classification task during prediction.

3.3 USE CASE DIAGRAM

This use case diagram illustrates role-based interaction within the industrial tool recognition system. The Admin manages dataset upload, CNN-based feature extraction using InceptionResNetV2, dataset splitting, and training of multiple classifiers including Decision Tree, KNN, Perceptron, and the proposed Hybrid DNN–Random Forest model, along with performance evaluation. The User role is restricted to uploading unseen tool images and receiving predicted tool categories, ensuring controlled system access aligned with real-world industrial deployment.

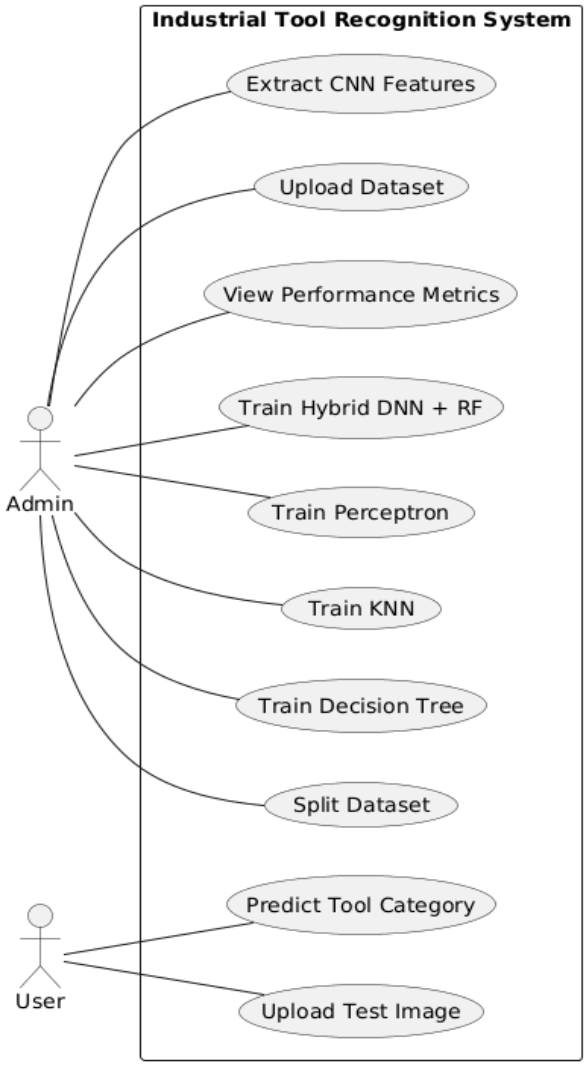


Figure 3.2: Use Case Diagram

3.4 CLASS DIAGRAM

The class diagram represents the modular architecture of the system. The GUI class controls user interactions, while DatasetManager handles dataset loading and class discovery. FeatureExtractor employs InceptionResNetV2 to generate deep feature vectors. Multiple classifiers inherit from a common Classifier structure. The HybridModel integrates DNN-generated probability vectors with a Random Forest classifier, forming the proposed ensemble approach. Database class supports authentication and role management.

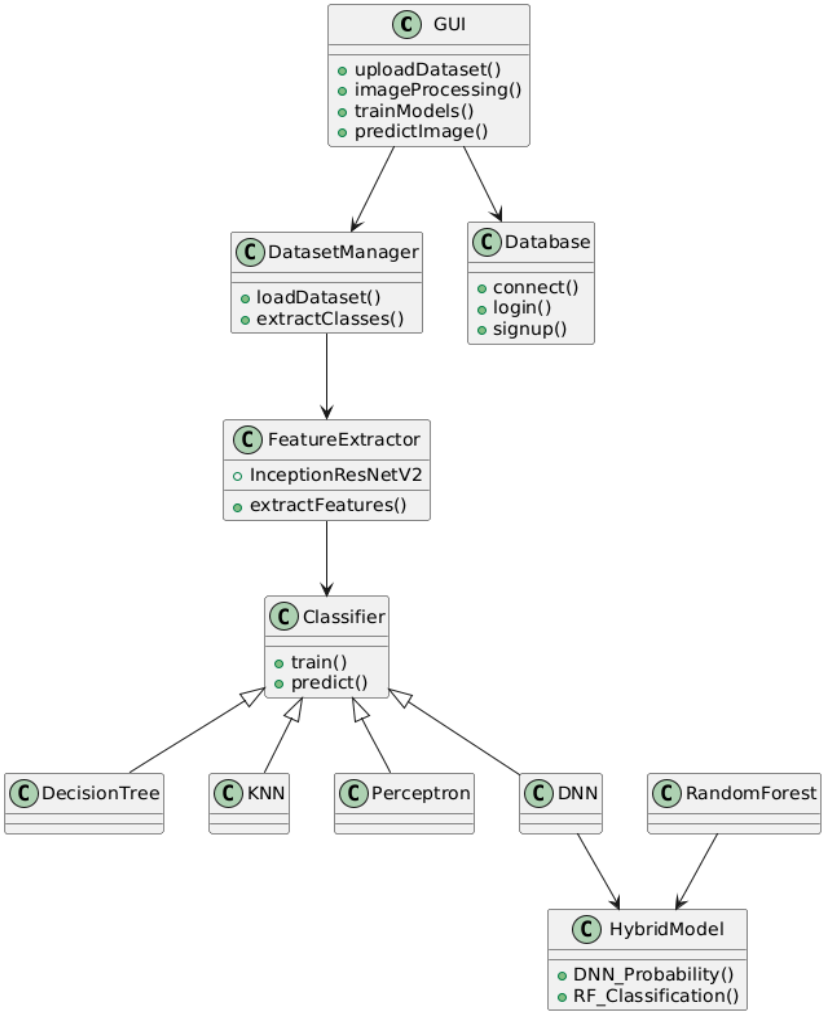
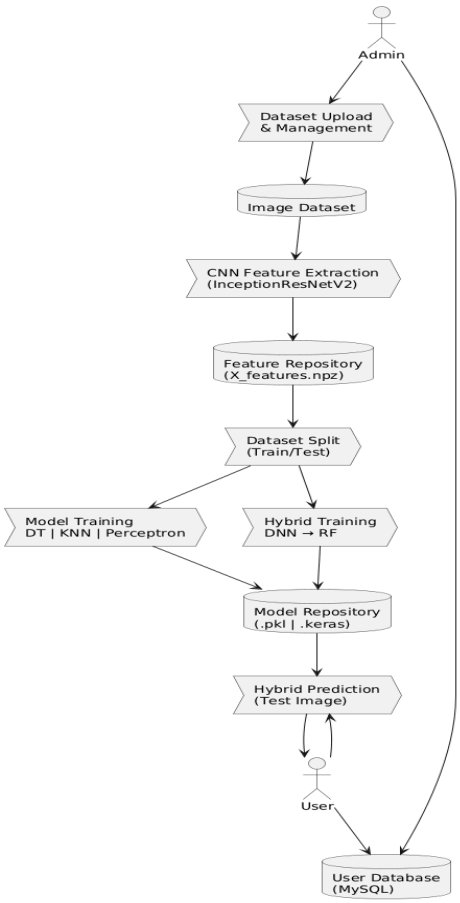


Figure 3.3: Class Diagram

3.5 DATA FLOW DIAGRAM

The data flow diagram represents the movement of data throughout the system. The Admin uploads an image dataset, which is stored and processed by the CNN feature extraction module using InceptionResNetV2 to generate deep feature vectors. These features are split into training and testing subsets and used to train both existing classifiers and the proposed hybrid DNN–Random Forest ensemble. All trained models are stored in a model repository for persistent access. During inference, a User uploads a test image, which follows the hybrid prediction pipeline to produce the final industrial tool classification. User authentication and access control are managed separately through the MySQL database.



-

Figure 3.4: Dataflow Diagram

3.6 ACTIVITY DIAGRAM

The activity diagram depicts the operational workflow of the system. After dataset upload and feature extraction, the workflow branches into either training existing classifiers or the proposed hybrid ensemble model. Model evaluation and persistence follow training. Finally, the system allows users to upload test images and receive predicted industrial tool classes.

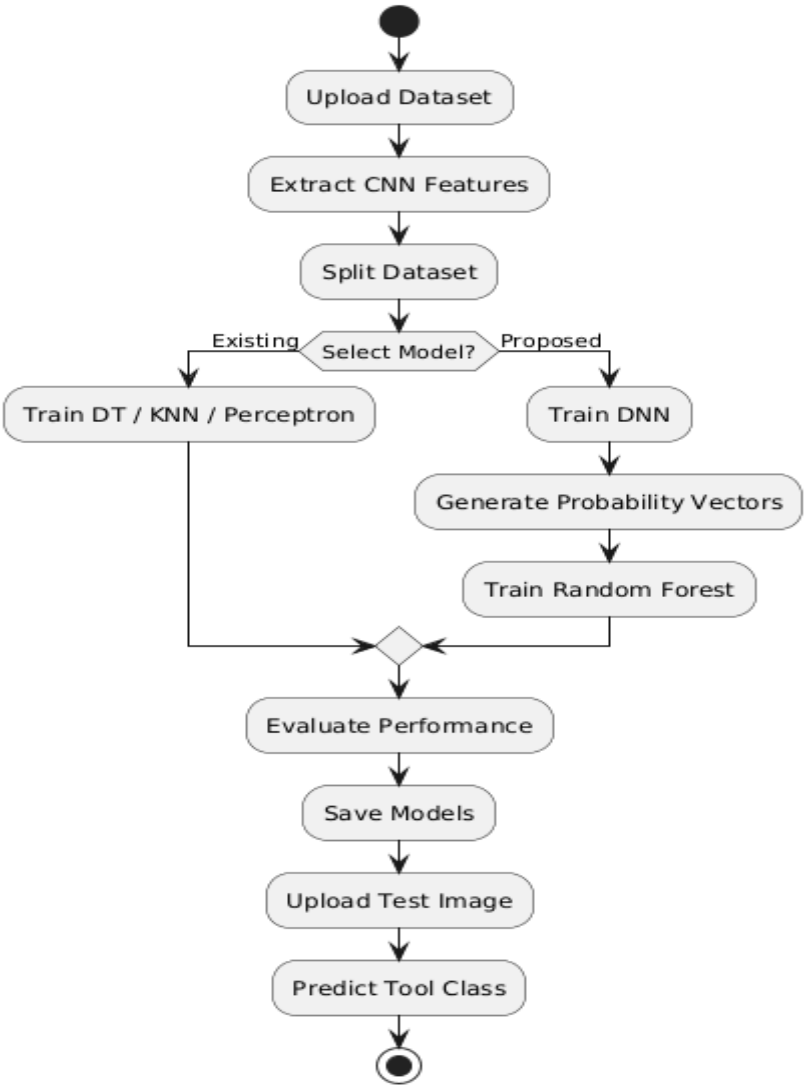


Figure 3.5: Activity Diagram

3.7 SEQUENCE DIAGRAM

This sequence diagram captures the temporal flow of training and prediction. During training, the Admin uploads the dataset, CNN extracts features, and multiple classifiers including the hybrid DNN–RF model are trained. During inference, the User uploads a test image, CNN features are extracted, DNN generates class probabilities, and Random Forest produces the final tool classification, which is displayed to the user.

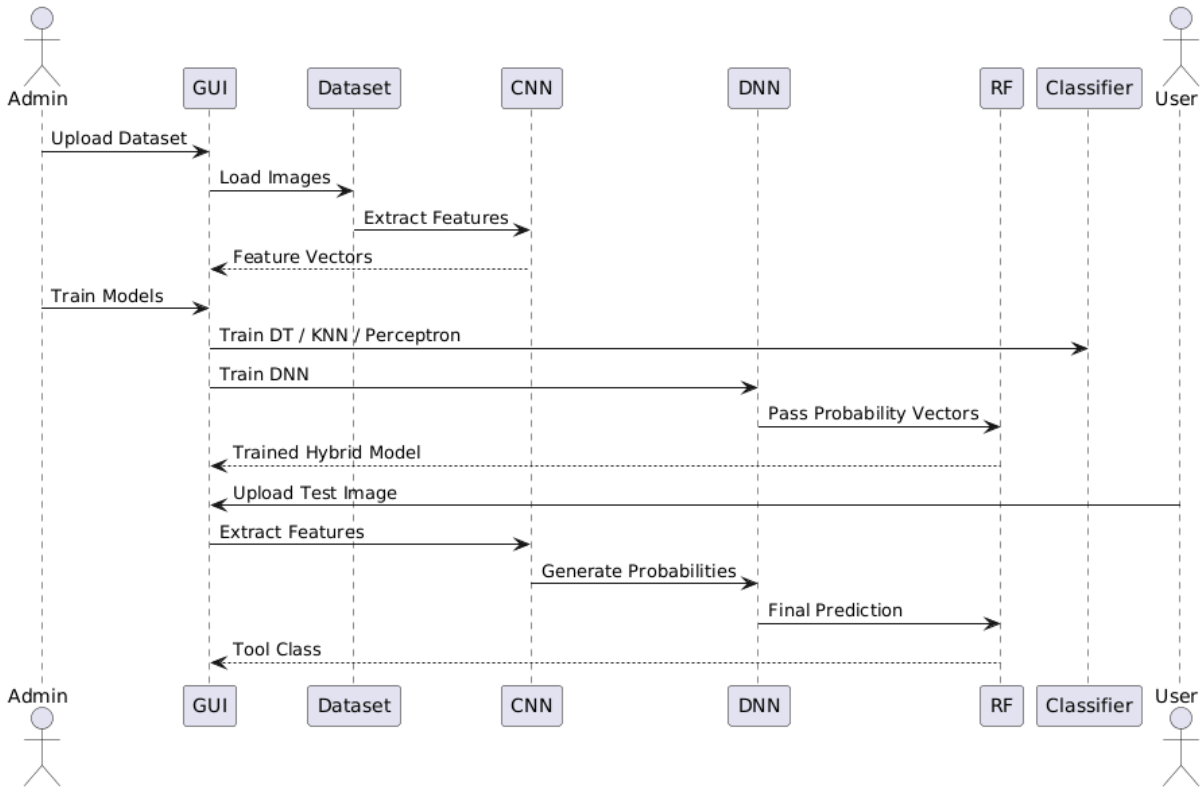


Figure 3.6: Sequence Diagram

4.IMPLEMENTATION

4. IMPLEMENTATION

The implementation of the proposed Industrial Tool Recognition System is carried out using image processing and machine learning techniques to automatically identify industrial tools from images. The system combines deep learning–based feature extraction with ensemble classification models to improve recognition accuracy. Initially, a dataset containing various industrial tool images such as hammers, screwdrivers, and other workshop tools is collected and organized into labeled folders representing different categories and subclasses. Before training the model, the images undergo preprocessing where they are resized to a fixed dimension of 128 × 128 pixels and normalized to ensure uniformity and better learning performance. After preprocessing, feature extraction is performed using a pretrained deep learning model called InceptionResNetV2, which extracts important visual features such as shapes, edges, and textures from the images. The base layers of this pretrained model are kept frozen to preserve the learned features, and the extracted features are flattened into a one-dimensional feature vector. These feature vectors are then divided into training and testing datasets, where approximately 80% of the data is used for training and 20% for testing. During the training phase, multiple classifiers such as Decision Tree, Perceptron, and a hybrid model combining Deep Neural Networks with Random Forest are trained to learn patterns from the extracted features. After training, the models are evaluated using performance metrics including Accuracy, F1-score, ROC curve, and Confusion Matrix to measure classification effectiveness. In the prediction phase, when a new industrial tool image is provided, it undergoes the same preprocessing and feature extraction steps, and the trained hybrid model predicts the tool category and subclass, after which the final classification result is displayed to the user.

4.1 ALGORITHMS USED

InceptionResNetV2 (Feature Extraction)

InceptionResNetV2 is a deep convolutional neural network architecture that combines the strengths of the Inception network and Residual connections. It is widely used for image recognition tasks because of its ability to extract complex visual features from images.

In this project, InceptionResNetV2 is used as a pretrained model to perform feature extraction. The convolutional layers of the model analyze the input image and detect patterns such as edges, textures, and shapes. The residual connections allow the network to train deeper architectures without suffering from the vanishing gradient problem.

By using this pretrained network, the system can efficiently extract meaningful features from tool images without requiring extensive training data. The extracted features are then converted into feature vectors and used as input for the classification algorithms.

Decision Tree Algorithm

The Decision Tree algorithm is a supervised machine learning method used for classification and prediction tasks. It works by splitting the dataset into different branches based on feature values.

In this algorithm, the model starts with a root node representing the entire dataset. The data is then divided into smaller subsets based on decision rules that maximize information gain. Each internal node represents a decision condition, while each leaf node represents the final classification result.

Decision Trees are easy to interpret and can handle both categorical and numerical data. In this project, the Decision Tree classifier learns decision rules based on extracted image features to classify industrial tools into their respective categories.

Perceptron Algorithm

The Perceptron is one of the simplest types of artificial neural networks used for binary classification problems. It works by computing a weighted sum of the input features and passing the result through an activation function to produce the output.

During training, the perceptron adjusts its weights based on the difference between predicted outputs and actual outputs. This learning process continues until the model achieves acceptable classification accuracy.

In this project, the perceptron classifier uses the extracted feature vectors from the deep learning model to learn patterns that differentiate one type of tool from another.

Random Forest Algorithm

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve classification accuracy. Instead of relying on a single decision tree, the algorithm creates many trees using different subsets of the dataset and features.

Each tree independently predicts the class of the input data, and the final prediction is obtained through majority voting among all trees. This approach reduces overfitting and improves model robustness.

In the proposed system, Random Forest helps improve tool recognition accuracy by combining the predictions of multiple trees trained on extracted image features.

Hybrid Classifier (DNN + Random Forest)

The proposed system introduces a hybrid classifier that integrates **Deep Neural Networks (DNN) with Random Forest** to achieve better classification performance.

The deep neural network processes the feature vectors and learns complex relationships between different features. The Random Forest component then aggregates predictions from multiple models to produce a more stable and accurate final prediction.

By combining deep learning with ensemble learning techniques, the hybrid classifier improves the system's ability to recognize industrial tools even in challenging workshop environments with variations in lighting, background, and tool orientation.

4.2 SAMPLE CODE

```
# Tkinter & GUI

import tkinter as tk

import tkinter

from tkinter import *

from tkinter import ttk, filedialog, messagebox, simpledialog

from PIL import Image, ImageTk

# Core Python

import os

import json

import pickle

import warnings

warnings.filterwarnings('ignore')

# Numerical & Data

import numpy as np

import pandas as pd

# Visualization

import matplotlib.pyplot as plt

import seaborn as sns

import cv2

# TensorFlow / Keras

from tensorflow.keras.models import Model, Sequential, load_model

from tensorflow.keras.layers import (

    Input, Dense, Dropout,
```

```
    Conv2D, MaxPooling2D, Flatten,
    GlobalAveragePooling2D
)
from tensorflow.keras.preprocessing import image
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.applications import InceptionResNetV2
from keras.callbacks import EarlyStopping
# Scikit-learn
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression, Perceptron
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.multioutput import MultiOutputClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix,
    roc_auc_score, roc_curve
)
from sklearn.preprocessing import label_binarize
```

```
# Model Persistence

import joblib

# Database

import pymysql

# Load InceptionResNetV2 base model

base_model = InceptionResNetV2(

    weights='imagenet',

    include_top=False,

    pooling='avg',

    input_shape=(128, 128, 3)

)

base_model.trainable = False

main = Tk('Ensemble Feature Learning for Accurate Recognition of Industrial Tools in Workshop Environments')

screen_width = main.winfo_screenwidth()

screen_height = main.winfo_screenheight()

# Set window size to full screen

main.geometry(f"{screen_width}x{screen_height}")

global filename

global X, Y

global model

global categories,model_folder

model_folder = "model"

import os

from tkinter import filedialog, END
```

```
def uploadDataset():
    global filename, categories
    text.delete('1.0', END)
    # select dataset folder
    filename = filedialog.askdirectory(initialdir=".")
    # class folders inside the dataset
    categories = [
        d for d in os.listdir(filename)
        if os.path.isdir(os.path.join(filename, d))
    ]
    categories.sort()
    text.insert(END, "Dataset loaded successfully\n")
    text.insert(END, f"Total classes: {len(categories)}\n")
    text.insert(END, f"Classes: {categories}\n")


def imageProcessing():
    global X, Y, categories, filename, base_model, model_folder
    text.delete('1.0', END)
    path = filename
    model_folder = "model"
    os.makedirs(model_folder, exist_ok=True)
    X_file = os.path.join(model_folder, "X_features.npz")
    Y_file = os.path.join(model_folder, "Y_labels.npy")
    classes_file = os.path.join(model_folder, "classes.json")
    if os.path.exists(X_file) and os.path.exists(Y_file):
```

```
        X = np.load(X_file)["X"]
        Y = np.load(Y_file)
        with open(classes_file, "r") as f:
            categories = json.load(f)
        text.insert(END, "Loaded features from model folder\n")
        return
    X, Y, categories = [], [], []
    print("Processing dataset from:", path)
    for class_name in sorted(os.listdir(path)):
        class_dir = os.path.join(path, class_name)
        if not os.path.isdir(class_dir):
            continue
        categories.append(class_name)
        label = len(categories) - 1
        for img_file in os.listdir(class_dir):
            if img_file.lower().endswith((".jpg", ".jpeg", ".png")):
                img_path = os.path.join(class_dir, img_file)
                img = image.load_img(img_path, target_size=(128, 128))
                arr = image.img_to_array(img) / 255.0
                arr = np.expand_dims(arr, axis=0)
                feat = base_model.predict(arr, verbose=0)
                X.append(np.squeeze(feat))
                Y.append(label)
    X = np.array(X)
    Y = np.array(Y)
```

```
    np.savez_compressed(X_file, X=X)
    np.save(Y_file, Y)
    with open(classes_file, "w") as f:
        json.dump(categories, f)
    text.insert(END, "Features extracted & saved in model folder\n")
    text.insert(END, f"Images: {len(X)} | Classes: {len(categories)}\n")


def Train_Test_split():
    global X, Y, x_train, x_test, y_train, y_test
    print("Splitting ->", X.shape, Y.shape)
    x_train, x_test, y_train, y_test = train_test_split(
        X, Y,
        test_size=0.20,
        random_state=42,
        stratify=Y
    )
    text.insert(END, f"Train samples: {x_train.shape}\n")
    text.insert(END, f"Test samples: {x_test.shape}\n")
def calculateMetrics(algorithm, categories, predict, y_test):
    a = accuracy_score(y_test, predict) * 100
    p = precision_score(y_test, predict, average='macro') * 100
    r = recall_score(y_test, predict, average='macro') * 100
    f = f1_score(y_test, predict, average='macro') * 100
    text.insert(END, algorithm + " Accuracy  :  " + str(a) + "\n")
    text.insert(END, algorithm + " Precision : " + str(p) + "\n")
```

```
    text.insert(END, algorithm + " Recall    : " + str(r) + "\n")

    text.insert(END, algorithm + " FScore    : " + str(f) + "\n")

    conf_matrix = confusion_matrix(y_test, predict)

    se = conf_matrix[0, 0] / (conf_matrix[0, 0] + conf_matrix[0, 1]) * 100 if conf_matrix.shape[0]
> 1 else 0

    sp = conf_matrix[1, 1] / (conf_matrix[1, 0] + conf_matrix[1, 1]) * 100 if conf_matrix.shape[0]
> 1 else 0

    text.insert(END, algorithm + ' Sensitivity : ' + str(se) + "\n")

    text.insert(END, algorithm + ' Specificity : ' + str(sp) + "\n\n")

    CR = classification_report(y_test, predict, target_names=categories)

    text.insert(END, algorithm + ' Classification Report \n')

    text.insert(END, algorithm + str(CR) + "\n\n")

    # Confusion Matrix Plot

    plt.figure(figsize=(6, 6))

    ax = sns.heatmap(conf_matrix, xticklabels=categories, yticklabels=categories,

                     annot=True, cmap="magma", fmt="g")

    ax.set_ylim([0, len(categories)])

    plt.title(algorithm + " Confusion matrix")

    plt.ylabel('True class')

    plt.xlabel('Predicted class')

    plt.show()


    # ROC Curve & AUC Score

    if len(categories) > 2:

        y_test_bin = label_binarize(y_test, classes=list(range(len(categories))))
```

```
        pred_bin = label_binarize(predict, classes=list(range(len(categories))))
        auc_score = roc_auc_score(y_test_bin, pred_bin, average="macro", multi_class="ovr")
        text.insert(END, algorithm + ' AUC Score (Macro Avg) : ' + str(auc_score * 100) + "\n\n")
        plt.figure(figsize=(8, 6))
        for i in range(len(categories)):
            fpr, tpr, _ = roc_curve(y_test_bin[:, i], pred_bin[:, i])
            plt.plot(fpr, tpr, label=f"{categories[i]} (AUC = {roc_auc_score(y_test_bin[:, i],
pred_bin[:, i]):.2f})")
        plt.plot([0, 1], [0, 1], 'k--')
        plt.xlabel("False Positive Rate")
        plt.ylabel("True Positive Rate")
        plt.title(algorithm + " ROC Curve (Multi-class)")
        plt.legend()
        plt.grid(True)
        plt.show()
    else:
        try:
            auc_score = roc_auc_score(y_test, predict)
            text.insert(END, algorithm + ' AUC Score : ' + str(auc_score * 100) + "\n\n")
            fpr, tpr, _ = roc_curve(y_test, predict)
            plt.figure()
            plt.plot(fpr, tpr, color='darkorange', lw=2,
                     label='ROC curve (area = %0.2f)' % auc_score)
            plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
            plt.xlabel('False Positive Rate')
```

```
        plt.ylabel('True Positive Rate')
        plt.title(algorithm + " ROC Curve")
        plt.legend(loc="lower right")
        plt.grid(True)
        plt.show()
    except ValueError:
        text.insert(END, algorithm + " AUC not available for this binary prediction.\n\n")


def Hybrid_DNN_RF_Model():
    global x_train, x_test, y_train, y_test, model_folder, categories
    text.delete('1.0', END)
    dnn_dir = os.path.join(model_folder, "DNN_prob_model_tf")
    rf_file = os.path.join(model_folder, "RF_on_DNN_Probs.pkl")
    x_train_flat = x_train.reshape((x_train.shape[0], -1))
    x_test_flat  = x_test.reshape((x_test.shape[0], -1))
    num_classes = len(np.unique(y_train))


    # -------- LOAD / TRAIN DNN --------
    if os.path.exists(dnn_dir):
        dnn = load_model(dnn_dir, compile=False)
        text.insert(END, "Loaded DNN (SavedModel)\n")
    else:
        inputs = Input(shape=(x_train_flat.shape[1],))
        x = Dense(128, activation='relu')(inputs)
        x = Dropout(0.6)(x)
```

```
    outputs = Dense(num_classes, activation='softmax')(x)
    dnn = Model(inputs, outputs)
    dnn.compile(optimizer="adam",
                loss="sparse_categorical_crossentropy",
                metrics=["accuracy"])
    dnn.fit(x_train_flat, y_train,
            epochs=5, batch_size=64,
            validation_split=0.1, verbose=1)
    dnn.save(dnn_dir)
    text.insert(END, "DNN trained & saved\n")

# -------- DNN → probability features --------
train_probs = dnn.predict(x_train_flat, verbose=0)
test_probs  = dnn.predict(x_test_flat,  verbose=0)

# -------- LOAD / TRAIN RF --------
if os.path.exists(rf_file):
    rf = joblib.load(rf_file)
    text.insert(END, "Loaded RF Hybrid\n")
else:
    rf = RandomForestClassifier(
        n_estimators=600,
        max_features="sqrt",
        class_weight="balanced_subsample",
        n_jobs=-1
```

```
        )
        rf.fit(train_probs, y_train)
        joblib.dump(rf, rf_file)
        text.insert(END, "RF trained & saved\n")

    # -------- Evaluate --------
    y_pred = rf.predict(test_probs)
    calculateMetrics("Hybrid (DNN Probs → RF)", categories, y_pred, y_test)

def predict_Hybrid_DNN_RF():
    global model_folder, categories, base_model
    text.delete('1.0', END)
    dnn_dir = os.path.join(model_folder, "DNN_prob_model_tf")
    rf_file = os.path.join(model_folder, "RF_on_DNN_Probs.pkl")
    if not os.path.exists(dnn_dir):
        text.insert(END, "DNN model not found. Train Hybrid model first.\n")
        return

    if not os.path.exists(rf_file):
        text.insert(END, "RF model not found. Train Hybrid model first.\n")
        return
    from tensorflow.keras.models import load_model
    from tensorflow.keras.preprocessing import image
    dnn = load_model(dnn_dir, compile=False)
    rf  = joblib.load(rf_file)
```

```
filename = filedialog.askopenfilename(initialdir="testImages")
if not filename:
    text.insert(END, "No image selected.\n")
    return

# ---- Preprocess image ----
img = image.load_img(filename, target_size=(128, 128))
arr = image.img_to_array(img) / 255.0
arr = np.expand_dims(arr, axis=0)

# ---- CNN features ----
cnn_feat = base_model.predict(arr, verbose=0).reshape(1, -1)

# ---- DNN probs → RF ----
probs = dnn.predict(cnn_feat, verbose=0)
y_pred = rf.predict(probs)[0]
class_name = categories[y_pred]

# ---- Show result in text box ----
text.insert(END, f"Hybrid Prediction: {class_name}\n")
], columns=['Metric', 'Model', 'Score'])
# Pivot for bar plot
pivot_df = df.pivot(index='Metric', columns='Model', values='Score')

# Graph properties
```

```
    plt.title('Comparison of Classifier Performance (KNN vs CNN+ETC)')
    plt.ylabel('Score (%)')
    plt.xticks(rotation=0)
    plt.ylim(0, 100)
    plt.legend(title='Model')
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.tight_layout()
    # Show the plot
    plt.show()
def close():
    main.destroy()


def setBackground():
    global bg_photo
    image_path = r"BG_image\images.jpeg"
    bg_image = Image.open(image_path)
    bg_image = bg_image.resize((screen_width, screen_height), Image.LANCZOS)
    #bg_image = bg_image.resize((900, 600), Image.LANCZOS)
    bg_photo = ImageTk.PhotoImage(bg_image)
   s bg_label = Label(main, image=bg_photo)
    bg_label.place(relwidth=1, relheight=1)
setBackground()
def connect_db():
    return pymysql.connect(host='localhost', user='root', password='root', database='sparse_db')
# Signup Functionality
```

```
def signup(role):
    def register_user():
        username = username_entry.get()
        password = password_entry.get()


# Login Functionality
def login(role):
    def verify_user():
        username = username_entry.get()
        password = password_entry.get()
        if username and password:
            try:
                conn = connect_db()
                cursor = conn.cursor()
                query = "SELECT * FROM users WHERE username=%s AND password=%s AND role=%s"
                cursor.execute(query, (username, password, role))
                if result:


                    messagebox.showinfo("Success", f"{role} Login Successful!")
                    login_window.destroy()
                    if role == "Admin":
                        show_admin_buttons()
                    elif role == "User":
                        show_user_buttons()
```

```
            else:
                messagebox.showerror("Error", "Invalid Credentials!")
        except Exception as e:
            messagebox.showerror("Error", f"Database Error: {e}")
    else:


        messagebox.showerror("Error", "Please enter all fields!")
    login_window = tk.Toplevel(main)
    login_window.geometry("400x300")
    login_window.title(f"{role} Login")
    Label(login_window, text="Username").pack(pady=5)
    username_entry = tk.Entry(login_window)
    username_entry.pack(pady=5)
    Label(login_window, text="Password").pack(pady=5)
    password_entry = tk.Entry(login_window, show="*")
    password_entry.pack(pady=5)
    tk.Button(login_window, text="Login", command=verify_user).pack(pady=10)


# Clear buttons function
def clear_buttons():
    for widget in main.place_slaves():
        if isinstance(widget, tkinter.Button):
            widget.destroy()


#  New Logout button
```

```
    tk.Button(main, text="Logout", command=show_login_screen, font=font1, bg="red").place(x=1100, y=600)

# User Button Functions

def show_user_buttons():

    font1 = ('times', 13, 'bold')

    clear_buttons()

    tk.Button(main, text="Prediction from Test Image",

              command=predict_Hybrid_DNN_RF,

              font=font1).place(x=550, y=200)

    tk.Button(main, text="Exit", command=close, font=font1).place(x=980, y=200)


 # New Logout button

    tk.Button(main, text="Logout", command=show_login_screen, font=font1, bg="red").place(x=1100, y=600)

def show_login_screen():

    clear_buttons()

    font1 = ('times', 14, 'bold')

    tk.Button(main, text="Admin Signup", command=lambda: signup("Admin"), font=font1, width=20, height=1, bg='Lightpink').place(x=100, y=100)

    tk.Button(main, text="User Signup", command=lambda: signup("User"), font=font1, width=20, height=1, bg='Lightpink').place(x=400, y=100)

    tk.Button(main, text="Admin Login", command=lambda: login("Admin"), font=font1, width=20, height=1, bg='Lightgreen').place(x=700, y=100)

    tk.Button(main, text="User Login", command=lambda: login("User"), font=font1, width=20, height=1, bg='Lightgreen').place(x=1000, y=100)
```

```
# Admin and User Buttons

font1 = ('times', 14, 'bold')

tk.Button(main, text="Admin Signup", command=lambda: signup("Admin"), font=font1, width=20, height=1, bg='Lightpink').place(x=100, y=100)

tk.Button(main, text="User Signup", command=lambda: signup("User"), font=font1, width=20, height=1, bg='Lightpink').place(x=400, y=100)

admin_button = tk.Button(main, text="Admin Login", command=lambda: login("Admin"), font=font1, width=20, height=1, bg='Lightgreen')

admin_button.place(x=700, y=100)

user_button = tk.Button(main, text="User Login", command=lambda: login("User"), font=font1, width=20, height=1, bg='Lightgreen')

user_button.place(x=1000, y=100)

main.mainloop()
```

5.RESULTS
&
DISCUSSION

5. RESULTS & DISCUSSION

5.1 Uploading dataset:

The figure shows the Admin interface confirming successful dataset loading with detected tool classes and options to proceed with feature extraction, dataset splitting, and model training.

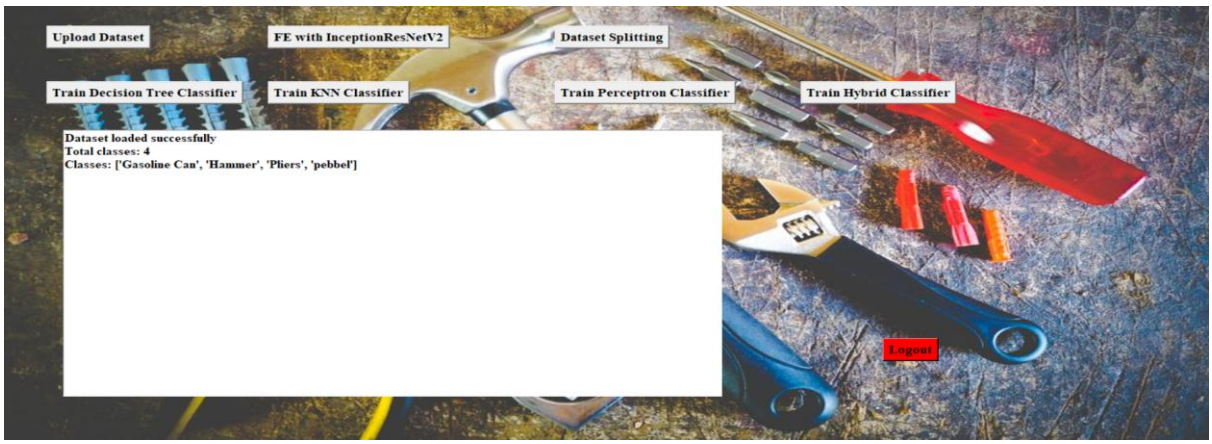


Figure 5.1 : Uploading dataset interface after admin login/signup

5.2 Feature Extraction:

The figure shows the successful feature extraction using the InceptionResNetV2 model, where the system loads or generates deep image features from the dataset and prepares them for dataset splitting and classifier training.

Figure 5.2 : InceptionResNetV2 feature extraction completed.

5.3 Using Decision Tree Classifier:

The figure shows the Decision Tree classifier results with ROC curves and a confusion matrix, indicating moderate classification performance and some misclassifications among similar industrial tool classes.

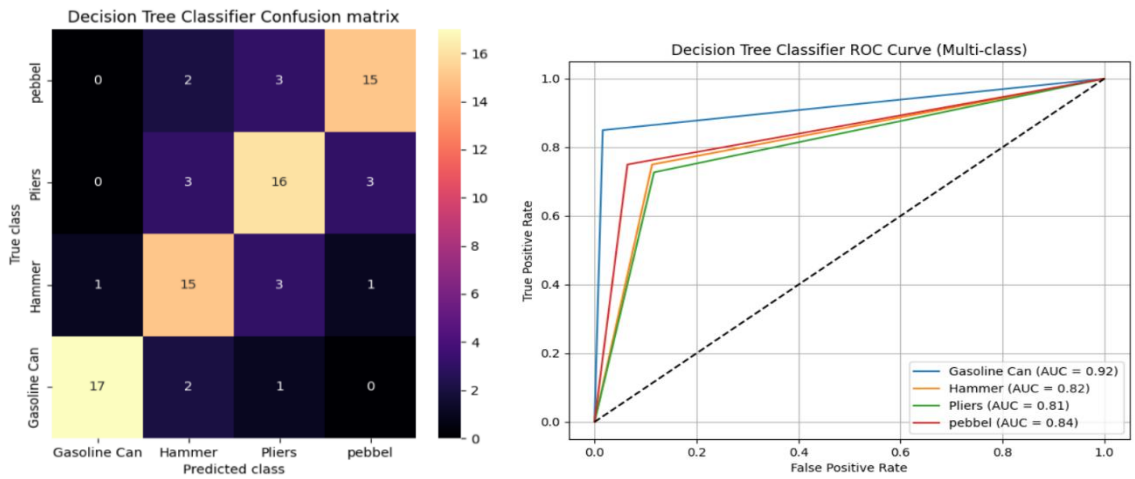


Figure 5.3 : Confusion matrix and ROC Curve obtained using DTC

5.4 Using K- Nearest Neighbour:

The figure shows the KNN classifier performance with a confusion matrix and ROC curves, demonstrating high accuracy and strong class separability using CNN-extracted features for industrial tool recognition.

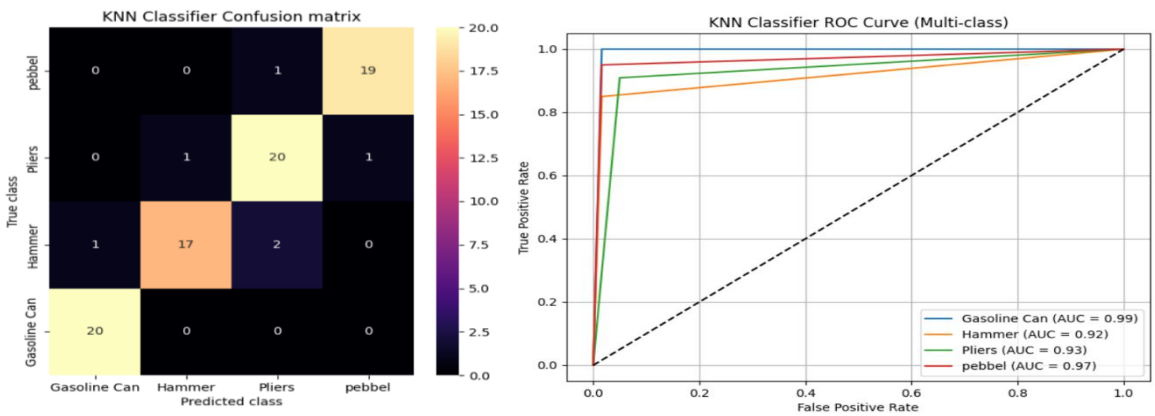


Figure 5.4 : Confusion matrix and ROC Curve obtained using KNN

5.5 Using Perceptron Classifier:

The figure shows the Perceptron classifier performance using a confusion matrix and ROC curves, indicating good recognition for Gasoline Can but moderate classification accuracy for other tool classes due to linear model limitations.

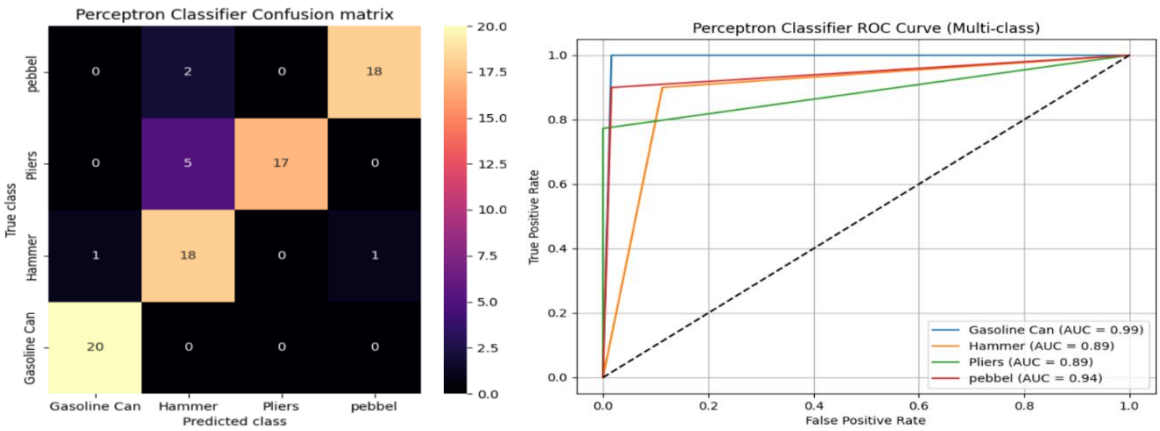


Figure 5.5 : Confusion matrix and ROC Curve obtained using perceptron classifier

5.6 Using DNN & RF Classifier:

The figure shows the Hybrid DNN with Random Forest model performance using a confusion matrix and ROC curves, demonstrating highly accurate classification and superior tool recognition compared to other classifiers.

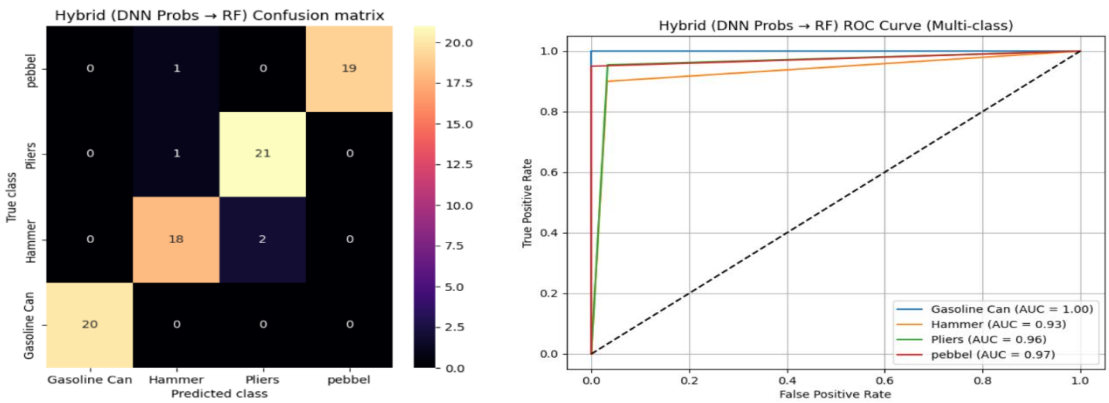


Figure 5.6 : Confusion matrix and ROC Curve obtained using Hybrid DNN with RF classifier

5.7 Prediction on Test Data:

The figure shows prediction results on unseen images where the Hybrid DNN with Random Forest model correctly identifies tools such as pliers and hammer, demonstrating accurate real-world tool recognition.

Figure 5.7: Prediction on test data using hybrid DNN with RF model

6.VALIDATION

6. VALIDATION

The validation of this project primarily relies on extensive testing and well-defined test cases to ensure the accuracy and effectiveness of the inappropriate content detection system. The testing process involves multiple stages, including dataset validation, model performance evaluation, and real-world testing. By implementing a structured validation approach, we can ensure that the system consistently delivers high accuracy in detecting inappropriate content while minimizing false positives and false negatives.

6.1 INTRODUCTION

Validation is an important stage in this project as it evaluates the effectiveness and reliability of the proposed Industrial Tool Recognition System. The validation process ensures that the trained models are capable of correctly identifying different industrial tools from unseen images and can generalize well beyond the training dataset. In this project, validation is performed by testing the trained classifiers on a separate test dataset that was not used during the training phase. Various performance metrics such as Accuracy, Precision, Recall, F1-score, ROC curve, and Confusion Matrix are used to measure the classification performance of the models. The validation results also help compare the performance of different classifiers such as Decision Tree, KNN, Perceptron, and the proposed Hybrid DNN with Random Forest model. Through this evaluation, the proposed hybrid approach demonstrates improved accuracy and better class discrimination, confirming its suitability for real-world industrial tool recognition applications.

6.2 TEST CASES

TABLE 6.2.1 : UPLOADING DATASET

Test case ID	Test case name	Purpose	Test Case	Output
1	Upload Dataset	To load the industrial tool image dataset into the system	The admin selects and uploads the dataset folder containing tool images	Dataset successfully loaded
2	Detect Tool Classes	To verify whether the system detects tool categories	The uploaded dataset contains folders like Hammer, Pliers, Gasoline Can, Pebble	Tool classes detected and displayed
3	Dataset Validation	To check whether the dataset format is correct	The system checks image formats and folder structure	Dataset validated successfully

TABLE 6.2.2 : CLASSIFICATION

Test case ID	Test case name	Purpose	Input	Output
1	Tool Classification Test 1	To check whether the classifier correctly identifies the tool	Image of Hammer is selected	Predicted Output: Hammer
2	Tool Classification Test 2	To check classification of another tool image	Image of Pliers is selected	Predicted Output: Pliers
3	Tool Classification Test 3	To test recognition of Gasoline Can	Image of Gasoline Can is selected	Predicted Output: Gasoline Can
4	Tool Classification Test 4	To verify classification accuracy for Pebble	Image of Pebble is selected	Predicted Output: Pebble

TABLE 6.2.3 : PREDICTION OF INDUSTRIAL TOOL

Test case ID	Test case name	Purpose	Input	Output
1	Tool Prediction Test 1	To verify whether the system predicts the tool correctly from a new image	New image of Hammer is uploaded	System predicts: Hammer
2	Tool Prediction Test 2	To check prediction for another tool category	New image of Pliers is uploaded	System predicts: Pliers
3	Tool Prediction Test 3	To test prediction accuracy for Gasoline Can	New image of Gasoline Can is uploaded	System predicts: Gasoline Can
4	Tool Prediction Test 4	To check prediction for Pebble class	New image of Pebble is uploaded	System predicts: Pebble

7.CONCLUSION
&
FUTURE ASPECTS

7. CONCLUSION & FUTURE ASPECTS

In conclusion, the project has successfully achieved its objectives, showcasing significant progress and outcomes. The implementation and execution phases were meticulously planned and executed, leading to substantial improvements and insights. Looking ahead, the future aspects of the project hold immense potential. Future developments will focus on expanding the scope, integrating new technologies, and enhancing sustainability. These advancements will not only strengthen the existing framework but also open new avenues for growth and innovation, ensuring the project remains relevant and impactful in the long term. This strategic approach will drive continuous improvement and success.

7.1 PROJECT CONCLUSION

The project successfully presents an ensemble feature learning framework for accurate recognition of industrial tools in workshop environments, integrating deep learning and classical machine learning techniques into a unified system. A pre-trained InceptionResNetV2 model is effectively utilized for robust feature extraction, transforming raw tool images into highly discriminative representations that capture complex visual patterns. Multiple baseline classifiers, including Decision Tree, KNN, and Perceptron, are implemented to establish comparative performance benchmarks, revealing the limitations of standalone models in handling inter-class similarity and environmental variability. To overcome these challenges, a proposed Hybrid DNN with Random Forest model is developed, where a DNN learns class probability distributions and a Random Forest refines final decisions through ensemble learning. Extensive experimental evaluation using confusion matrices, ROC–AUC curves, and statistical performance metrics demonstrates that the hybrid approach consistently outperforms existing methods, achieving superior accuracy, precision, recall, and F-score. The system is further strengthened by a Tkinter-based GUI and role-based authentication, enabling secure dataset management, model training, and real-time prediction. The results validate that combining deep CNN features with probabilistic and ensemble learning significantly enhances robustness, generalization, and reliability, making the proposed framework highly suitable for intelligent automation and safety-oriented applications in industrial workshop environments.

7.2 FUTURE ASPECTS

Although the proposed Industrial Tool Recognition System achieves good accuracy in identifying industrial tools, several improvements can be made in the future to enhance its performance and usability. In future work, the system can be extended to recognize a larger variety of industrial tools and equipment by training the model with a bigger and more diverse dataset. The accuracy can also be further improved by integrating more advanced deep learning architectures and ensemble learning techniques.

Another possible enhancement is the development of a real-time recognition system using live camera input, which can automatically detect and classify tools in workshops or manufacturing environments. The system can also be integrated with Industrial IoT (IIoT) platforms to monitor tool usage, track tool conditions, and support smart manufacturing processes. Additionally, deploying the model as a mobile or web-based application would allow users to easily identify tools using smartphones or other portable devices. These improvements can make the system more practical, scalable, and suitable for real-world industrial applications.

8.BIBLIOGRAPHY

8. BIBLIOGRAPHY

8.1 REFERENCES

[1] Schafer Tesch da Silva, F.; André da Costa, C.; Paredes Crovato, C.D.; da Rosa Righi, R. Looking at energy through the lens of Industry 4.0: A systematic literature review of concerns and challenges. CAIE 2020, 143, 106426.

[2] Mourtzis, D.; Vlachou, E.; Milas, N. Industrial big data as a result of IoT adoption in manufacturing. Procedia CIRP 2016, 55, 290–295.

[3] Gajate, A.; Bustillo, A.; Haber, R.E. Transductive neurofuzzy-based torque control of a milling process: Results of a case study. Int. J. Innov. Comput. Inf. Control 2012, 8, 3495–3510.

[4] Ebrahimi, M.; Victory, J.L. Web-based machine tool condition monitoring. In Network Intelligence: Internet-based Manufacturing, Proceedings of the SPIE, Boston, MA, USA, 29 December 2000; International Society for Optics and Photonics: Bellingham, WA, USA, 2000.

[5] Benardos, P.G.; Vosniakos, G.-C. Predicting surface roughness in machining: A review. Int. J. Mach. Tool. Manuf. 2003, 43, 833–844.

[6] Garrido-Labrador JL, Puente-Gabarri D, Ramírez-Sanz JM, Ayala-Dulanto D, Maudes J. Using Ensembles for Accurate Modelling of Manufacturing Processes in an IoT Data-Acquisition Solution. *Applied Sciences*. 2020; 10(13):4606. https://doi.org/10.3390/app10134606

[7] Odeh A, Abu Taleb A. Ensemble-Based Deep Learning Models for Enhancing IoT Intrusion Detection. *Applied Sciences*. 2023; 13(21):11985. https://doi.org/10.3390/app132111985

[8] Goodarzi P, Schauer J, Schütze A. Robust Distribution-Aware Ensemble Learning for Multi-Sensor Systems. *Sensors*. 2025; 25(3):831. https://doi.org/10.3390/s25030831

[9] Jurek A, Nugent C, Bi Y, Wu S. Clustering-Based Ensemble Learning for Activity Recognition in Smart Homes. *Sensors*. 2014; 14(7):12285-12304. https://doi.org/10.3390/s140712285

[10] Yücesoy E. Automatic Age and Gender Recognition Using Ensemble Learning. *Applied Sciences*. 2024; 14(16):6868. https://doi.org/10.3390/app14166868

[11] Lai W-I, Chen Y-Y, Sun J-H. Ensemble Machine Learning Model for Accurate Air Pollution Detection Using Commercial Gas Sensors. *Sensors*. 2022; 22(12):4393. https://doi.org/10.3390/s22124393

[12] Ma Y, Xu T, Han S, Kim K. Ensemble Learning of Multiple Deep CNNs Using Accuracy-Based Weighted Voting for ASL Recognition. *Applied Sciences*. 2022; 12(22):11766. https://doi.org/10.3390/app122211766

[13] Zhao H, Li C, Yin X, Li X, Zhou R, Fu R. Ensemble Learning-Enabled Security Anomaly Identification for IoT Cyber–Physical Power Systems. *Electronics*. 2022; 11(23):4043. https://doi.org/10.3390/electronics11234043

[14] Du K-L, Zhang R, Jiang B, Zeng J, Lu J. Foundations and Innovations in Data Fusion and Ensemble Learning for Effective Consensus. *Mathematics*. 2025; 13(4):587. https://doi.org/10.3390/math13040587

[15] Alshazly H, Linse C, Barth E, Martinetz T. Ensembles of Deep Learning Models and Transfer Learning for Ear Recognition. *Sensors*. 2019; 19(19):4139. https://doi.org/10.3390/s19194139

[16] Alshazly H, Linse C, Barth E, Martinetz T. Ensembles of Deep Learning Models and Transfer Learning for Ear Recognition. *Sensors*. 2019; 19(19):4139. https://doi.org/10.3390/s19194139

[17] de Zarzà I, de Curtò J, Hernández-Orallo E, Calafate CT. Cascading and Ensemble Techniques in Deep Learning. *Electronics*. 2023; 12(15):3354. https://doi.org/10.3390/electronics12153354

[18] Ku Abd. Rahim KN, Elamvazuthi I, Izhar LI, Capi G. Classification of Human Daily Activities Using Ensemble Methods Based on Smartphone Inertial Sensors. *Sensors*. 2018; 18(12):4132. https://doi.org/10.3390/s18124132

8.2 GITHUB LINK

https://github.com/SaiSamhithaKamuni/ENSEMBLE-FEATURE-LEARNING-FOR-ACCURATE-RECOGNITION-OF-INDUSTRIAL-TOOLS-IN-WORKSHOP-ENVIRONMENTS